Module: Distributed Clusters, Fault Tolerance, and MapReduce Programming

1. Laboratory Objective

In this comprehensive laboratory session, students will move beyond theoretical distributed systems and build a fully functional, multi-node Hadoop cluster using Docker. By the end of this lab, students will be able to:

- Deploy and orchestrate a multi-node distributed cluster utilizing Docker Compose.
- Simulate Real-World Failures to observe Hadoop's built-in fault tolerance and self-healing mechanisms.
- Develop Distributed MapReduce Applications using Python (Hadoop Streaming) to process diverse datasets.

2. Multi-Node Architecture & Docker Configuration

To simulate a real cluster on a single machine, we use Docker. Below is the explanation of the required configuration files used to spin up a custom multi-node environment.

2.1 The Environment File (.env)

This file passes critical configuration variables into the Docker containers at startup. This prevents us from having to manually edit XML files inside the containers.

```
CORE_CONF_fs_defaultFS=hdfs://namenode:9000
 HDFS_CONF_dfs_replication=2
 HDFS_CONF_dfs_namenode_datanode_registration_ip_hostname_check=false
```

Why we use these settings: The defaultFS tells all nodes where the NameNode is. We explicitly set the replication factor to 2 so that if a single DataNode crashes, our data survives on another node. We disable the hostname check because Docker dynamically assigns IPs, which would otherwise cause Hadoop security checks to fail.

2.2 Docker Compose Configuration (docker-compose.yml)

The Docker Compose file orchestrates our multi-node cluster. Notably:

- **Custom Builds:** Instead of pulling the image directly, we use `build: ./namenode` and `build: ./datanode` to point to our custom Dockerfiles that include Python.
- **Container Name Removal for DataNode:** We explicitly DO NOT set a `container_name` for the datanode service. This allows Docker Compose to dynamically spin up multiple replicas (e.g., datanode-1, datanode-2) without naming conflicts. If we hardcoded the name, scaling would crash.

- **Common Network:** All containers are explicitly connected to a common custom bridge network (`hadoop_net`). This ensures that the DataNodes and ResourceManager can seamlessly resolve the NameNode by its service name (`namenode`) via internal DNS without knowing its IP address.

```
version: "3"

 services:
   namenode:
   build: ./namenode
   container_name: namenode
   ports:
          - "9870:9870"
          - "9000:9000"
   env_file:
          - ./hadoop.env
   networks:
          - hadoop_net

   datanode:
   build: ./datanode
   depends_on:
          - namenode
   env_file:
          - ./hadoop.env
   networks:
          - hadoop_net

   resourcemanager:
   image: bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8
   container_name: resourcemanager
   ports:
          - "8088:8088"
   depends_on:
       - namenode
          - datanode
   env_file:
          - ./hadoop.env
   networks:
          - hadoop_net

 networks:
   hadoop_net:
   driver: bridge
```

2.3 The Custom Dockerfiles (NameNode & DataNode)

Hadoop natively uses Java, but modern data engineering heavily relies on Python. We extend the base images to include Python. Without this step, when YARN tries to run our python mapper/reducer on a DataNode, it will throw a 'command not found' error. Create this Dockerfile in both the `namenode` and `datanode` directories:

```
FROM bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
 # (Note: Use hadoop-namenode base image for the NameNode directory)
```

```
 USER root

 RUN sed -i \
   -e 's|deb.debian.org|archive.debian.org|g' \
   -e 's|security.debian.org|archive.debian.org|g' \
   -e '/stretch-updates/d' \
   /etc/apt/sources.list && \
   echo 'Acquire::Check-Valid-Until "false";' >
/etc/apt/apt.conf.d/99no-check-valid-until && \
   apt-get update && \
   apt-get install -y python3 python3-pip && \
   ln -sf /usr/bin/python3 /usr/bin/python && \
   rm -rf /var/lib/apt/lists/*
```

3. Cluster Initialization & Scaling

Task: Spin up the cluster. We will start 1 NameNode and dynamically scale our DataNodes to 3 to simulate a real distributed environment.

```
docker compose up -d --build --scale datanode=3
```

Task: Verify the containers are running.

```
docker ps
```

Task: Check Cluster Health. Access the NameNode and ask it for a status report. This confirms that all 3 DataNodes successfully sent their heartbeats over the internal Docker network.

```
docker exec -it namenode bash
 hdfs dfsadmin -report
```

4. HDFS Fault Tolerance & Data Recovery Simulation

Distributed systems expect hardware to fail. Hadoop handles this through block replication. If a node goes offline, the NameNode detects the missing heartbeat and instructs the remaining nodes to duplicate the vulnerable data blocks.

Task 4.1: Upload a file into HDFS (Run this inside the NameNode container)

```
echo "Hello Hadoop Cluster" > testfile.txt
 hdfs dfs -mkdir -p /data
 hdfs dfs -D dfs.replication=3 -put testfile.txt /data/
```

Task 4.2: Locate the blocks using File System Check (FSCK). This command queries the NameNode metadata to physically locate which DataNode containers hold your file's blocks.

```
hdfs fsck /data/testfile.txt -files -blocks -locations
```

Task 4.3: Simulate a Hardware Failure. Open a new terminal on your host machine and forcefully stop one of the DataNodes:

```
docker stop <container_name_of_datanode_2>
```

Task 4.4: Observe the Self-Healing. Go back to your NameNode terminal and run `hdfs dfsadmin -report`. You will temporarily see 'Dead datanodes (1)'. Shortly after, the NameNode will initiate auto-replication on the surviving nodes.

Task 4.5: Restore the Node

```
docker start <container_name_of_datanode_2>
```

5. Distributed MapReduce Laboratory Tasks

For these tasks, we will use Hadoop Streaming, allowing us to write distributed Mappers and Reducers in Python.

TASK 1: Sales Aggregation

Big Data Context: A common task in data warehousing is computing aggregate metrics (like Total Revenue) from massive, unstructured CSV dumps scattered across thousands of servers.

Input File Generation:

We use a bash 'Here-Document' (cat <<EOF) to quickly write multiple lines of mock transactional data directly into a text file without needing a text editor. Each line represents a transaction in a 'Product,Price' format.

```
cat <<EOF > sales.txt
 Laptop,1000
 Phone,500
 Laptop,1500
 Tablet,700
 Phone,200
 Monitor,300
 EOF
 hdfs dfs -put sales.txt /data/
```

Algorithm & Data Flow Execution:

- **Mapper:** The Mapper script reads the file line-by-line via Standard Input. It splits the string by the comma. It emits the product name as the key, and the price as the value, separated by a tab.
- **Shuffle & Sort Phase:** Before the Reducer starts, Hadoop intercepts the Mapper output. It automatically routes identical keys to the same destination over the network and sorts them (e.g., all "Laptop" keys arrive sequentially: `Laptop 1000`, then `Laptop 1500`).
- **Reducer:** Because the incoming data is strictly sorted, the Reducer logic is incredibly simple. It keeps a running sum. The moment the product name changes (e.g., from Laptop to Monitor), it prints the final sum for the previous product and resets the counter.

```
# mapper.py
 import sys
 for line in sys.stdin:
```

```
  product, price = line.strip().split(',')
    print(f"{product}\t{price}")

# reducer.py
import sys
current_product = None
current_sum = 0

for line in sys.stdin:
  product, price = line.strip().split('\t')
  price = int(price)

  if current_product == product:
        current_sum += price
  else:
        if current_product:
           print(f"{current_product}\t{current_sum}")
        current_product = product
        current_sum = price
if current_product:
    print(f"{current_product}\t{current_sum}")
```

TASK 2: Feature Statistics (Machine Learning Preprocessing)

Big Data Context: Before training ML models, data must be normalized. Finding the global Minimum, Maximum, and Mean across a distributed cluster requires funneling all records to a single aggregation point.

Input File Generation:

We use a bash `for` loop combined with the `$RANDOM` variable. This generates 1,000 random integers between 0 and 500, simulating a massive column of sensor readings, and appends them to `features.txt`.

```
for i in {1..1000}; do echo $((RANDOM % 500)) >> features.txt; done
 hdfs dfs -put features.txt /data/
```

Algorithm & Data Flow Execution:

- **Mapper:** To calculate a true global mean or global max, ALL data must be sent to a single Reducer. To force this behavior in Hadoop, the Mapper emits a hardcoded, static key (like "ALL_STATS") for every single number.
- **Reducer:** The Reducer receives millions of values all under the single key "ALL_STATS". It initializes running variables for `min`, `max`, `sum`, and `count`. It iterates through the input stream, updating these variables, and finally prints the aggregated results once the stream ends.

```
# mapper.py
 import sys
 for line in sys.stdin:
   value = line.strip()
     print(f"ALL_STATS\t{value}")
```

```
# reducer.py (Pseudocode Hint)
# Initialize count=0, total_sum=0, min_val=float('inf'), max_val=float('-inf')
# Loop through sys.stdin
# For each value, update min, max, count, and sum
# Print final aggregated stats (Min, Max, Mean)
```

TASK 3: Advanced Graph - Mutual Friend Recommendation

Big Data Context: Graph processing (like finding triangles or mutual connections) is notoriously difficult in distributed systems. MapReduce handles this by pairing vertices and grouping intersecting edges.

Input File Generation:

We generate an Adjacency List. Each line represents a user, followed by a colon, followed by a comma-separated list of their current direct friends.

```
cat <<EOF > friends.txt
A:B,C,D
B:A,C,E
C:A,B,D,E
D:A,C
E:B,C
EOF
hdfs dfs -put friends.txt /data/
```

Algorithm & Data Flow Execution:

- **Mapper:** The Mapper performs two crucial actions. First, it emits `(User-Friend, DIRECT)` to ensure we don't recommend someone who is already a friend. Second, it generates combinations of the friend list. For example, if A is friends with B and C, it implies B and C share A as a mutual friend. It emits `(B-C, A)`. CRITICAL: The pair (B-C) MUST be sorted alphabetically before emitting, ensuring both (B-C) and (C-B) resolve to the exact same Reducer.
- **Reducer:** The Reducer receives grouped pairs, e.g., `B-C: [A, DIRECT, E]`. If it sees "DIRECT" in the list, it aborts. Otherwise, the remaining values are the mutual friends!

```
# mapper.py (Hint)
# 1. Parse line into User and FriendsList
# 2. Loop FriendsList: print sorted pairs of User & Friend with value "DIRECT"
# 3. Use itertools.combinations to generate all unique pairs from FriendsList
# 4. Print sorted combination pair with the User as the value (the mutual link)

# reducer.py (Hint)
# Group incoming data by the Pair key.
# If the value "DIRECT" appears for that pair, skip/ignore it.
# Otherwise, accumulate the values into a list (these are the mutual friends).
# Print Pair -> Mutual Friends List
```